

Разработка API на Flask



АЛЕКСАНДР ПУШКАРЕВ

**Senior Software Engineer,
TripAdvisor**

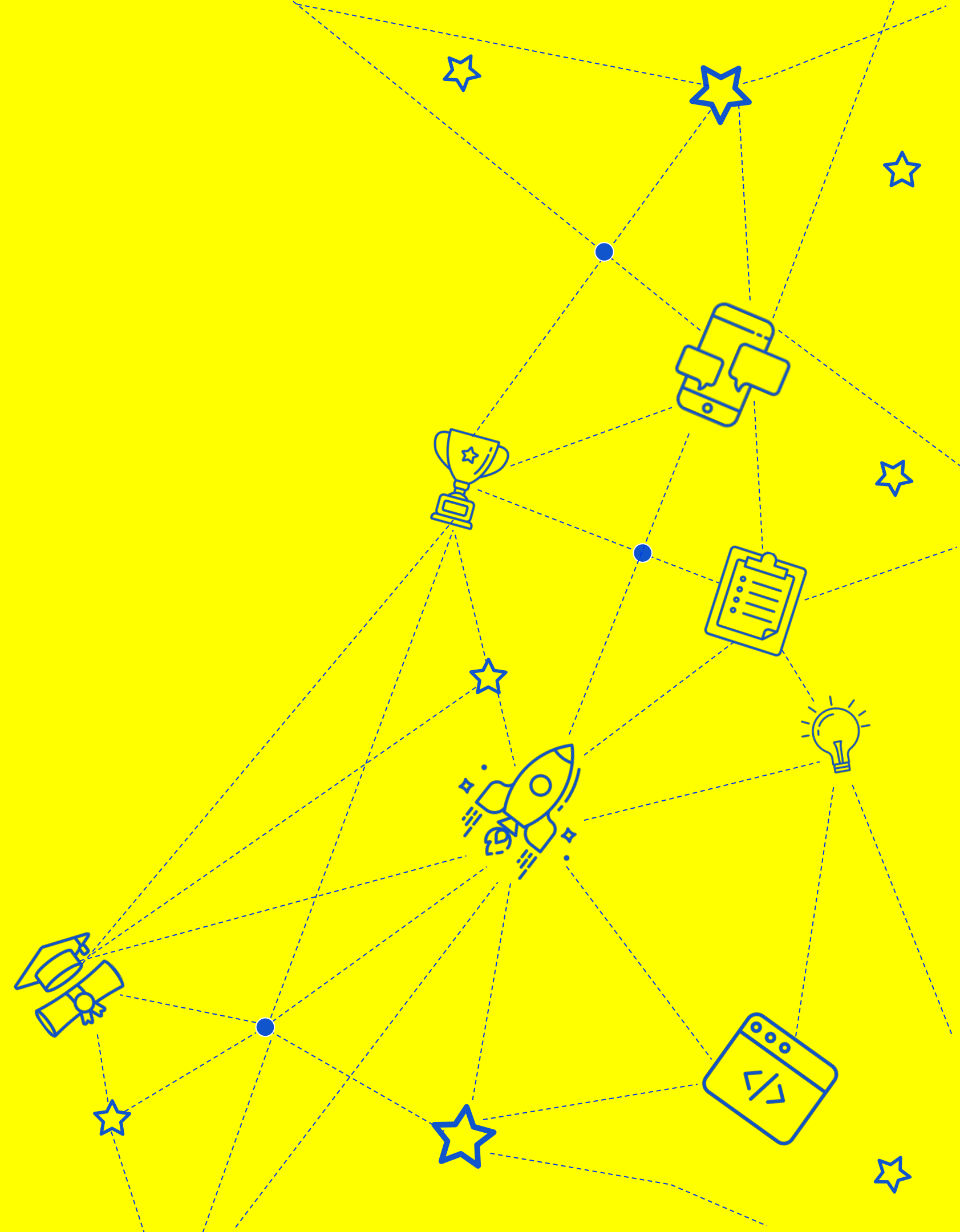
Xilinx (AMD), WorkFusion, EPAM
Systems, Toptal

- Опыт в IT – более 13 лет
- Автор нескольких курсов по разработке и тестирование ПО
- Keynote speaker на международных конференциях
- Специализируюсь на разработке и обеспечении качества ПО

Содержание урока

- ★ Что такое API?
- ★ Из чего состоит REST API?
- ★ Работа с сущностями. CRUD
- ★ Инструменты фреймворка Flask
необходимые для разработки API
- ★ Практическое применение всех знаний

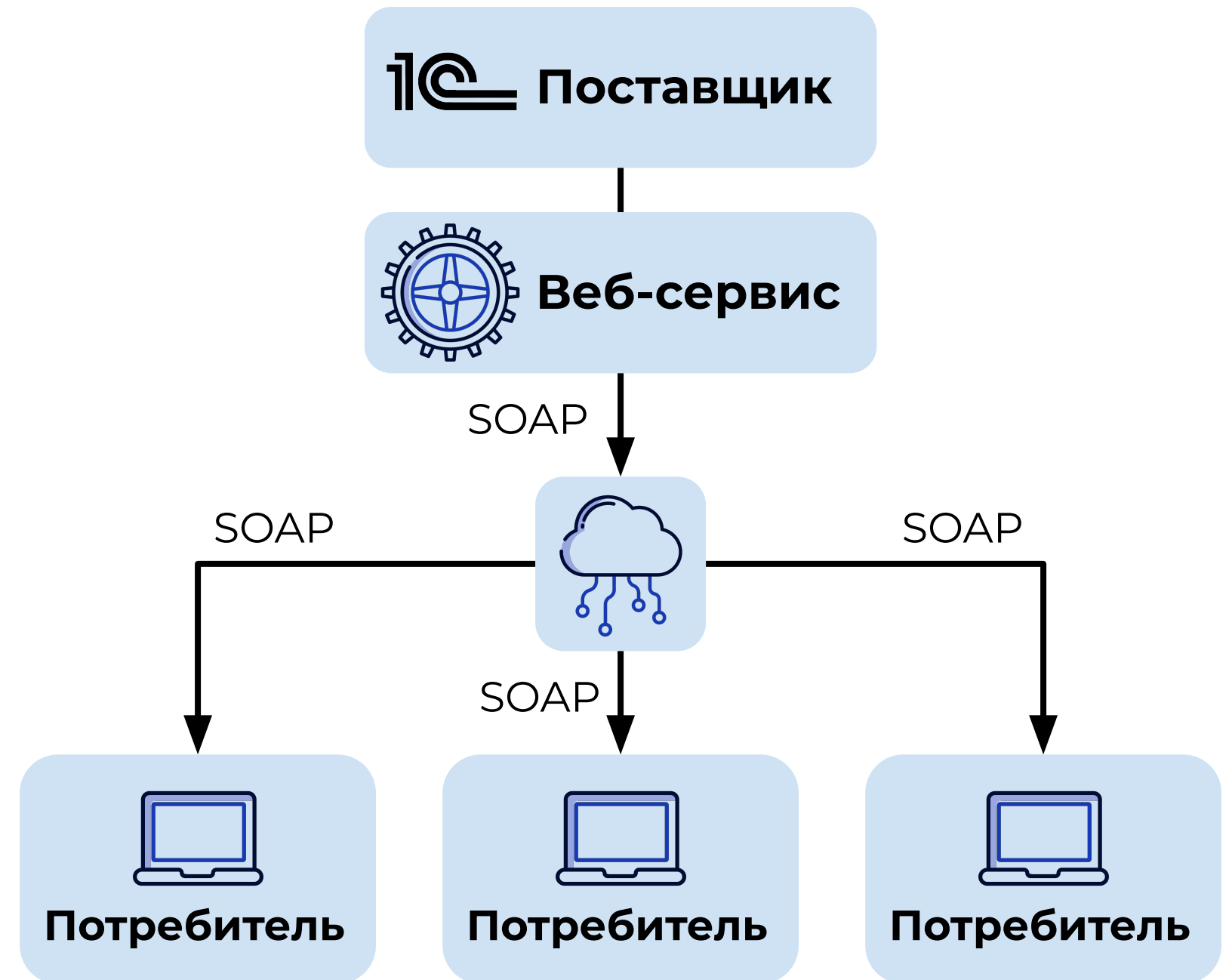
Что такое API?



Что такое API?

API – Application programming interface

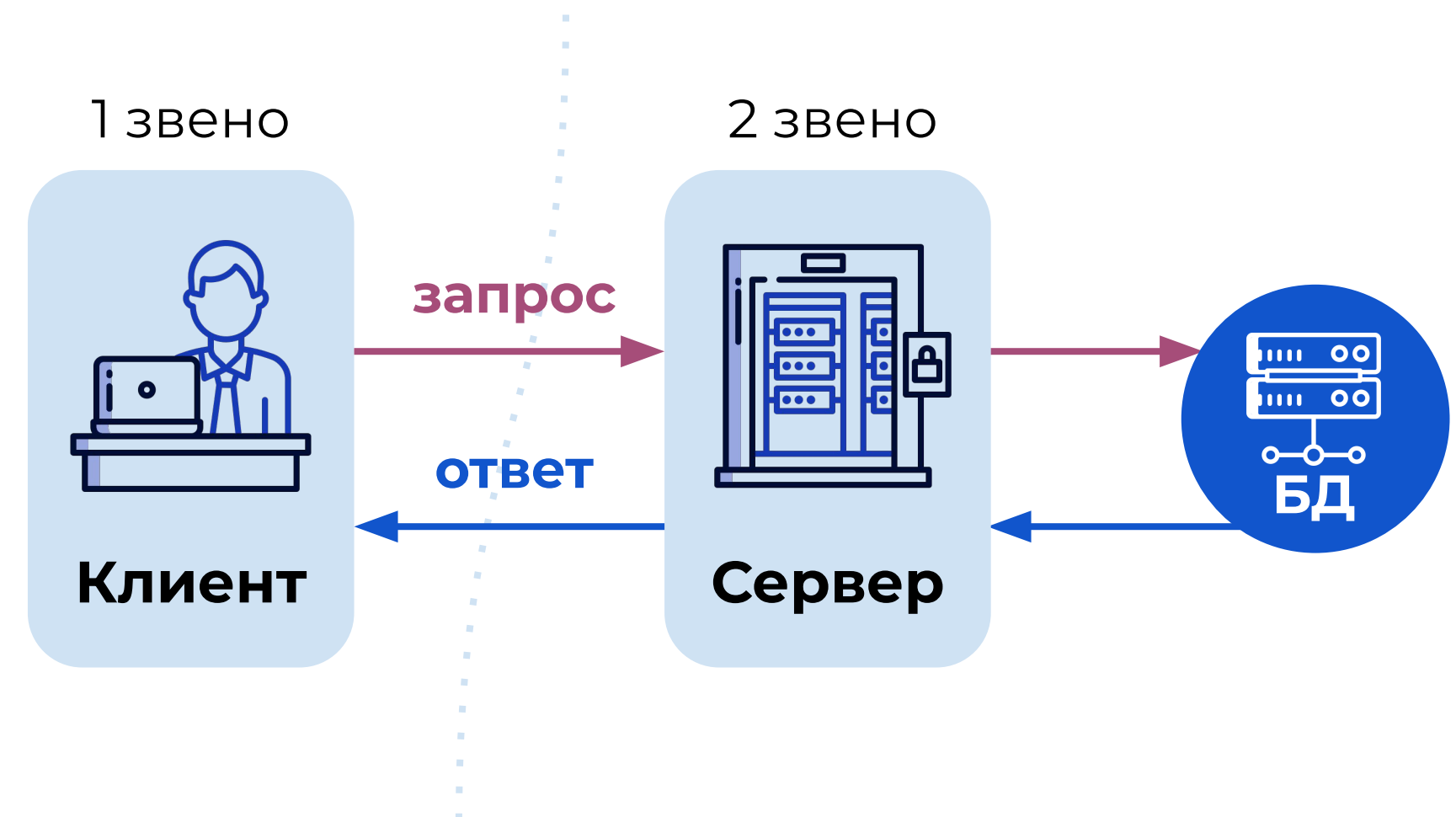
- ★ Задача API – предоставить интерфейс, удобный для компьютера
- ★ Клиент (Front-end) обращается к серверу (Back-end) посредством API



Клиент-сервер архитектура

Клиент-серверная архитектура – это наиболее простой способ программного сетевого взаимодействия.

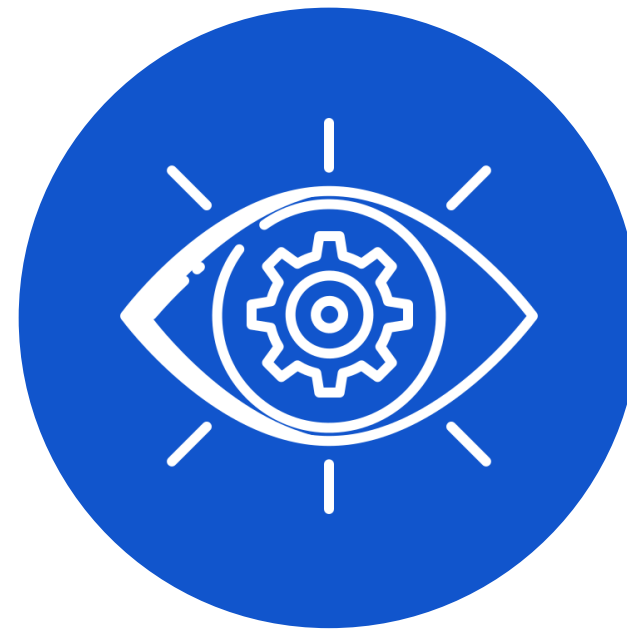
- ★ **Клиент** – программа, выполняемая на устройстве конечного потребителя. Многие параметры вне нашего контроля.
- ★ **Сервер** – программа, выполняемая на «наших» мощностях в контролируемой среде.



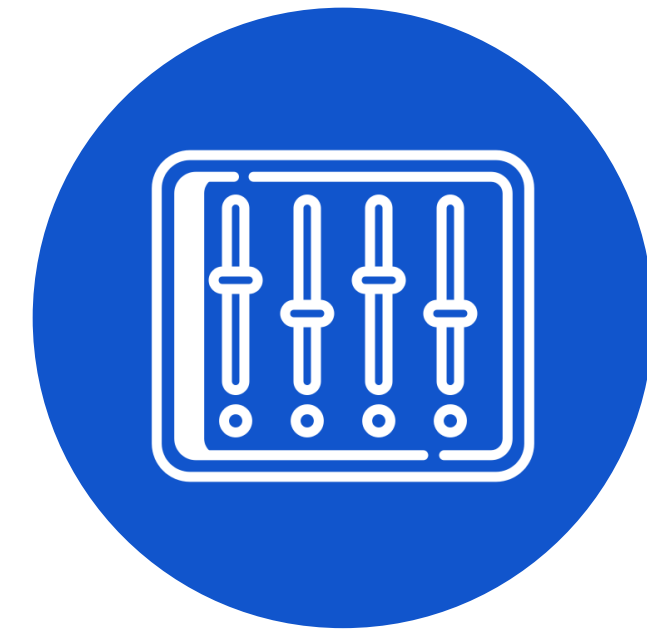
Плюсы архитектуры клиент-сервер



**Более быстрая
разработка
под различные
устройства (Android,
Iphone, etc)**



**Контролируемая
среда позволяет
нам гарантировать
производительность
и доступность сервера**

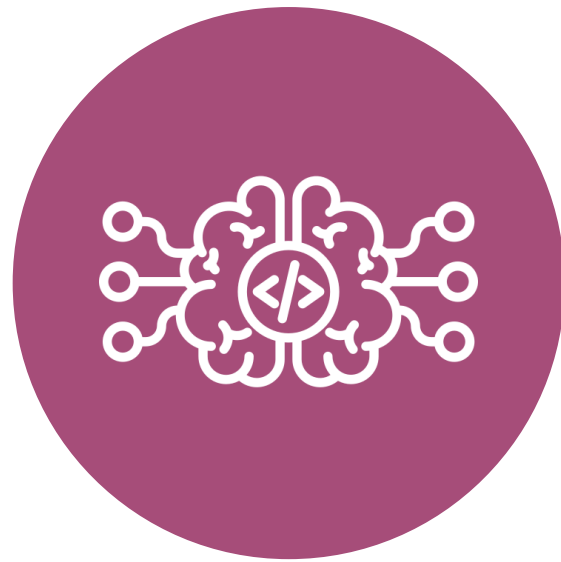


**Возможность
добавления
или изменения
функционала
без доп. действий
от пользователя**

Минусы архитектуры клиент-сервер



**Неизбежная
задержка из-
за сетевого
компонента**



**Необходимость
обработки сетевых
неполадок**



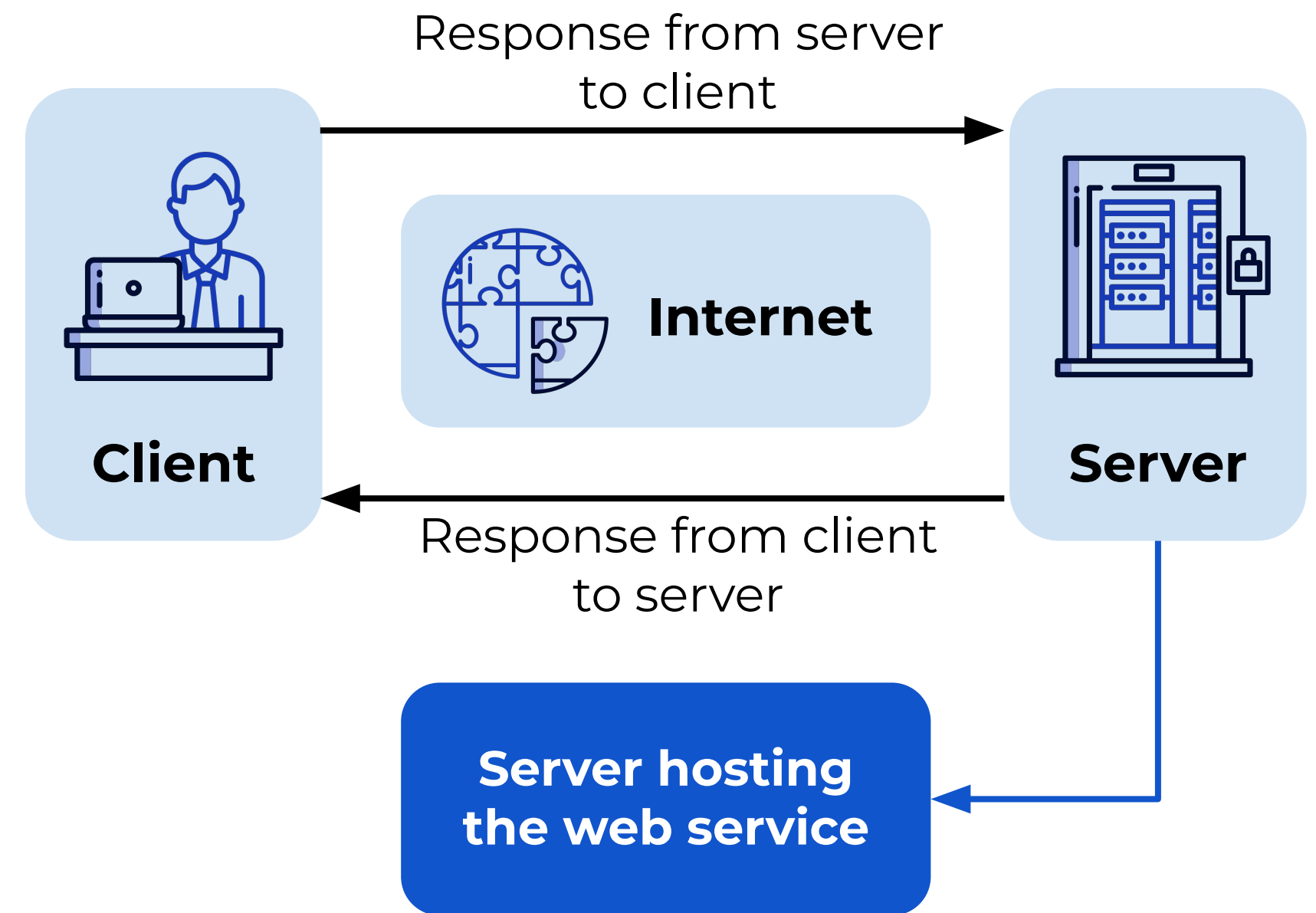
**Отсутствие
полного контроля
над средой клиента
приводит
к неизбежным
различиям UX**



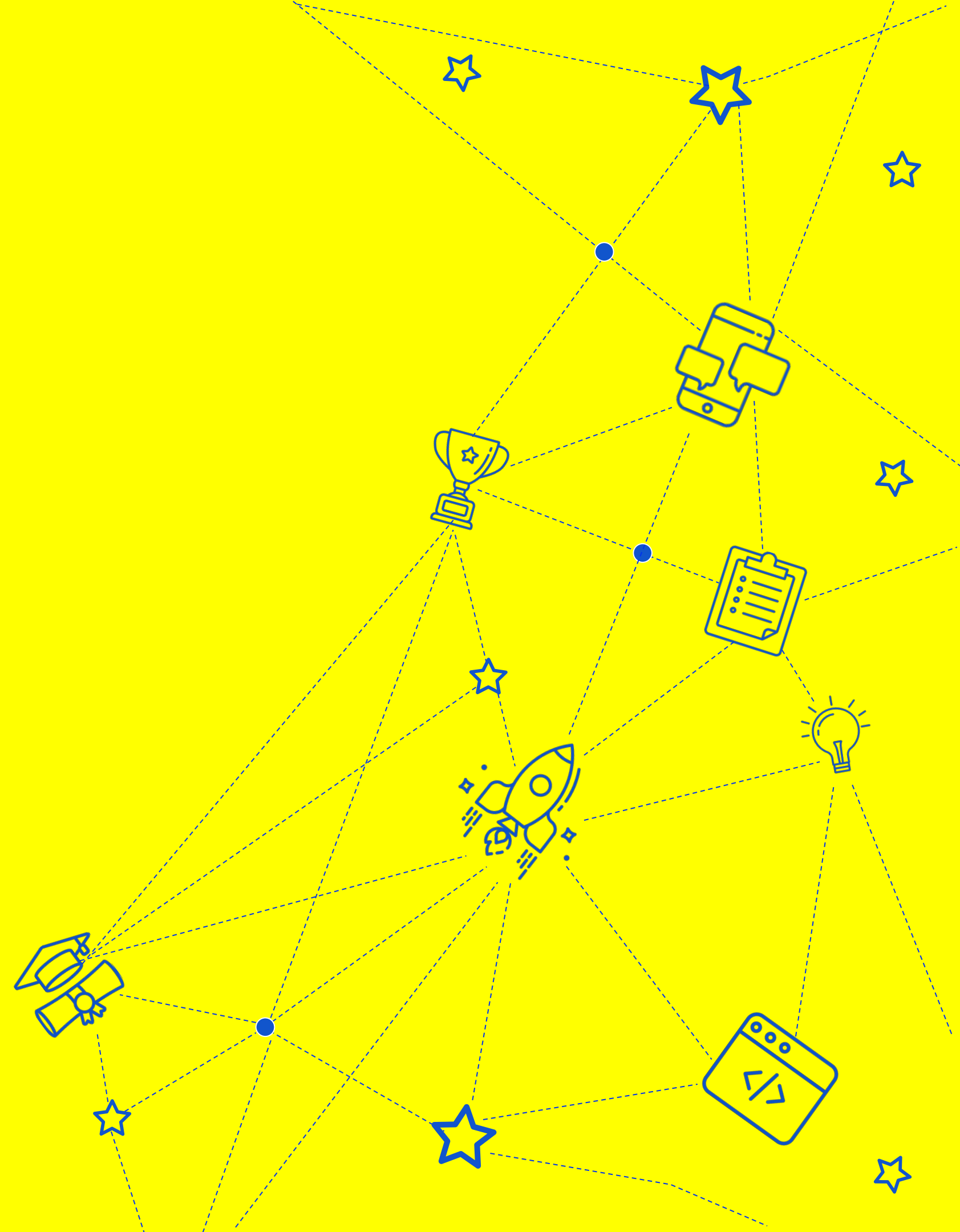
**Сетевое
взаимодействие
всегда
увеличивает
кибер риски**

Технологии построения API

- ★ Протокол передачи (HTTP)
- ★ Протокол кодировки данных (JSON\YAML/XML)
- ★ Опционально – протокол описания контракта (WSDL, Swagger)
- ★ Опционально – протокол построения самого API (SOAP\GraphQL)



Из чего состоит REST API?



REST API

REpresentational State Transfer

REST – термин предложенный для описания архитектуры построения WWW в докторской диссертации Роя Филдинга, одного из авторов протокола HTTP.

Архитектурный стиль построения API

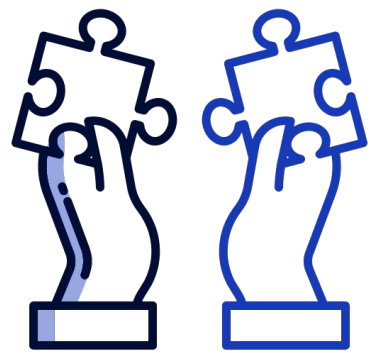
Основные идеи:

- ★ Клиент-серверное взаимодействие
- ★ Stateless
- ★ Понятие ресурса
- ★ Понятие состояния ресурса
- ★ Опора на существующие технологии вместо разработки новых



Основные компоненты REST API

REST API берут за основу подмножество протокола HTTP



Ресурс («сущность» или «noun»)

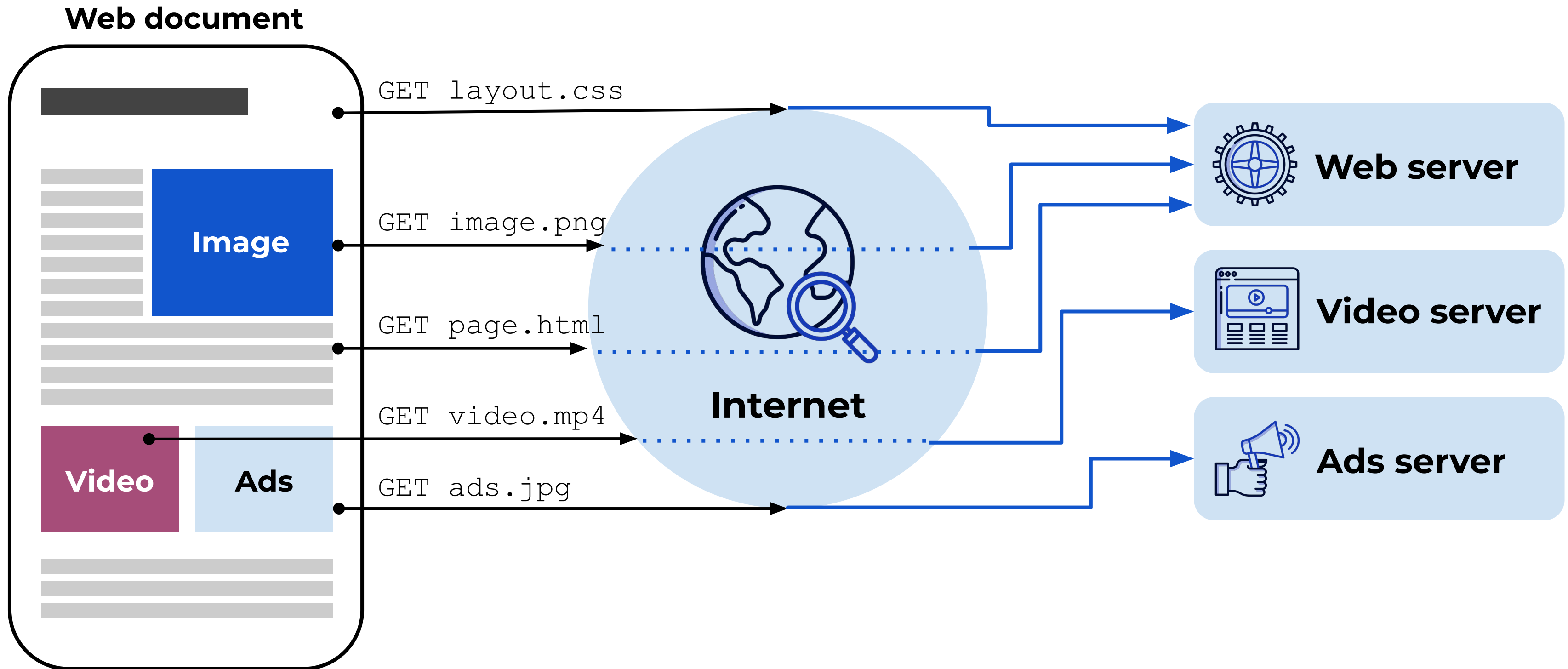
- ★ определяется URL
- ★ чаще всего кодируется с помощью JSON



Действие над ресурсом («Verb»)

- ★ определяется HTTP методом

HOW HTTP WORKS



REST or RESTful API?

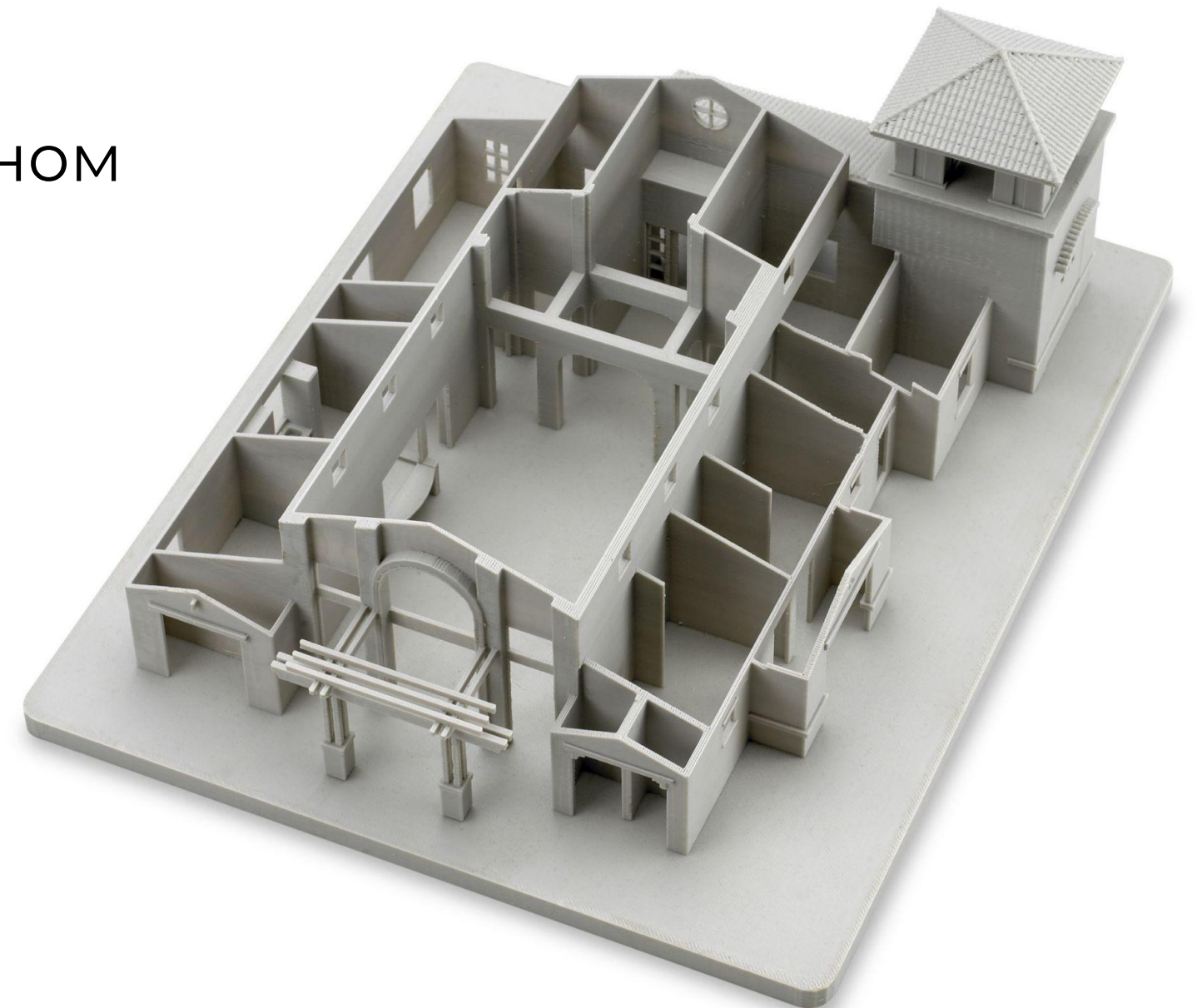
По сути – это одно и то же.

REST – название архитектурного стиля.

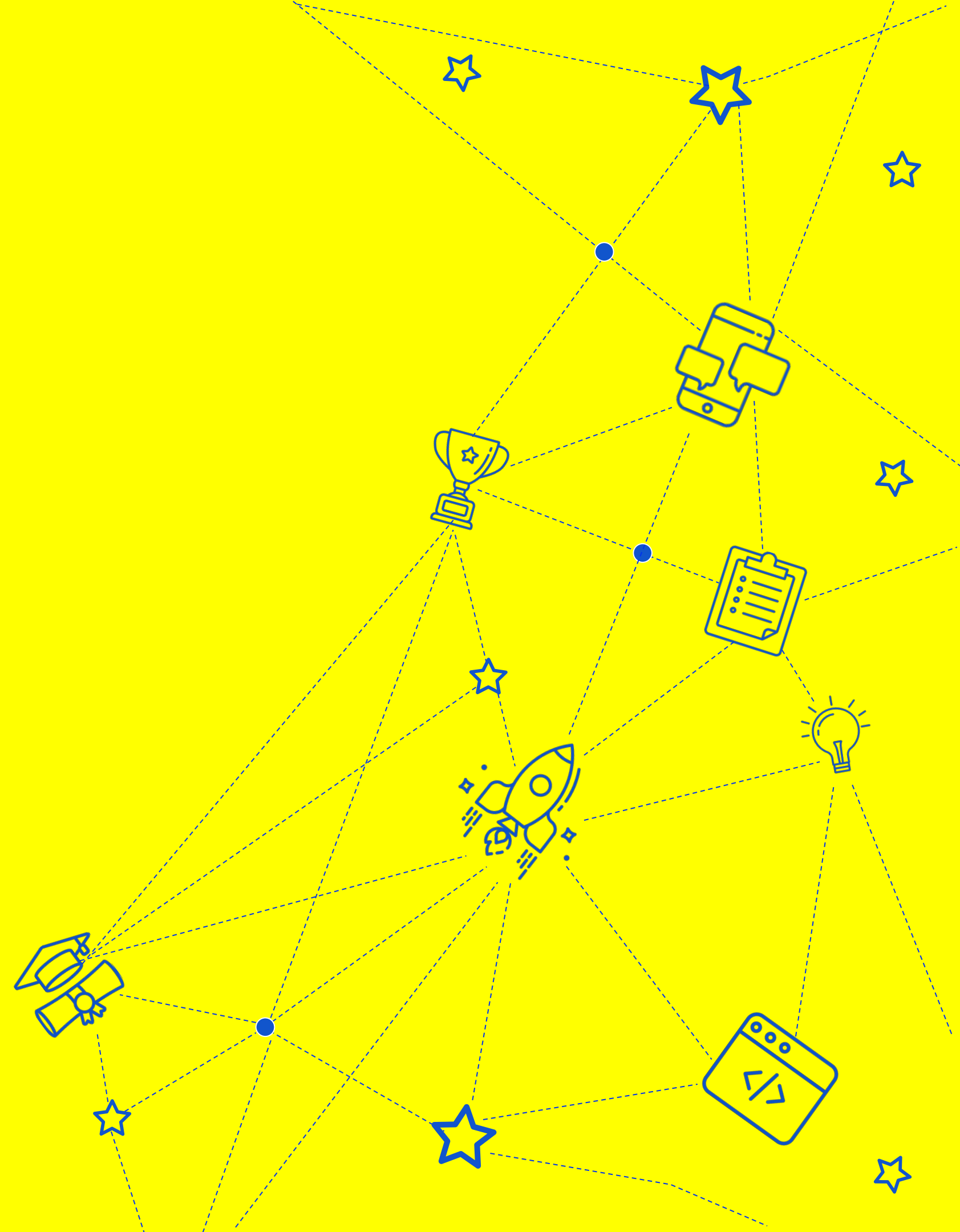
RESTful API означает, что API построено в полном соответствии со стилем REST.



**А значит, что таких API
не существует.**



Работа с сущностями. CRUD



Почему REST так популярен?

ресурс == сущность

Почти любую бизнес логику можно представить, как операции над некими сущностями.

Сущность это модель некого реального объекта или понятия:



Пользователь



Заказ



Товар

Почему REST так популярен?

сущность + действие над ресурсом
==
бизнес логика



Данный набор действий
является не только достаточным,
но и часто избыточным

Четыре базовых действия (CRUD):

CREATE

создание
сущности

RETRIEVE

чтение\доступ
к сущности

UPDATE

обновление\
изменение
сущности

DELETE

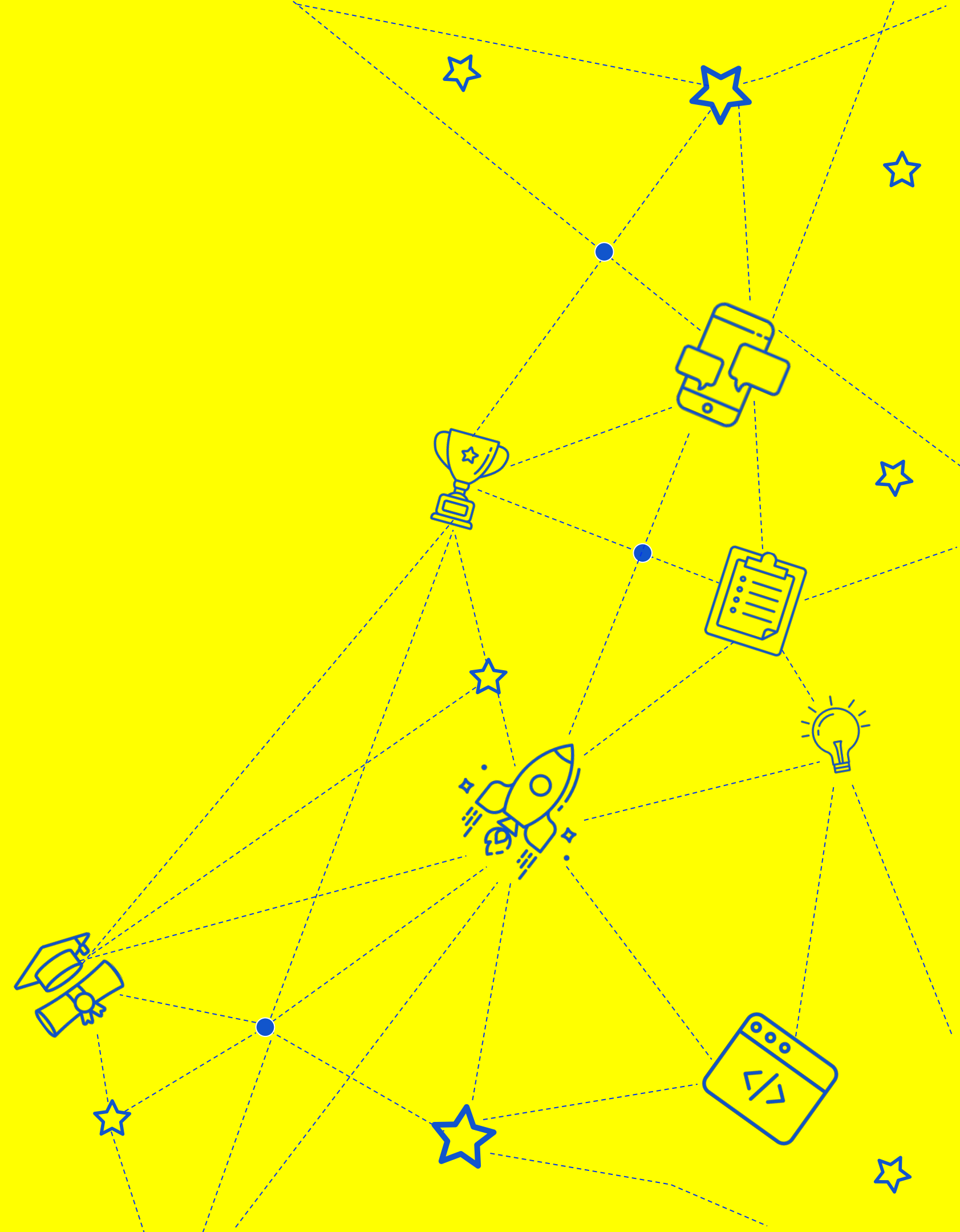
удаление
сущности

Почему REST так популярен?

- ☆ **Протокол HTTP** – самый популярный протокол передачи данных
- ☆ Протокол HTTP предоставляет возможности CRUD «из коробки»
- ☆ Ничего не надо делать нового, оно работает само.

HTTP Method	CRUD Action
GET	Retrieve a resource
POST	Create a resource
PUT	Update a resource
DELETE	Delete a resource

Инструменты фреймворка Flask необходимые для разработки API



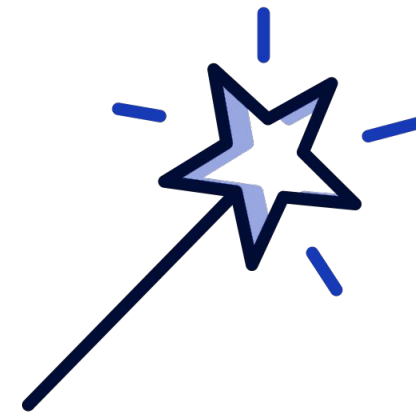
Работа с телом запроса

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api', methods=['POST'])
def parse_json():
    json_data = request.get_json()
    # Do something with the json_data
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run()
```



Тело запроса необходимо преобразовать в объект некоего класса или словарь

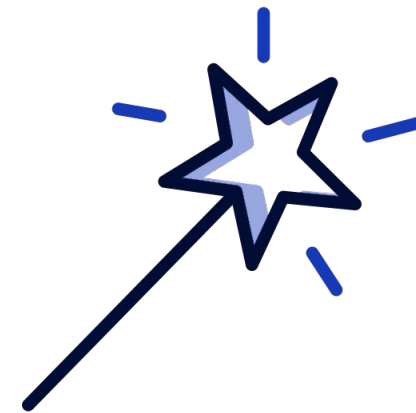
Работа с телом запроса

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api', methods=['POST'])
def parse_json():
    json_data = request.get_json()
    # Do something with the json_data
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run()
```



Тело запроса необходимо преобразовать в объект некоего класса или словарь

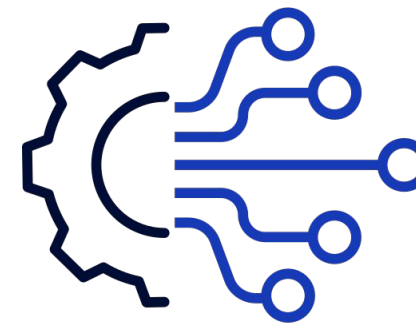
Работа с методом

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api', methods=['POST'])
def parse_json():
    json_data = request.get_json()
    # Do something with the json_data
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run()
```



Необходимо указать
ожидаемый метод запроса

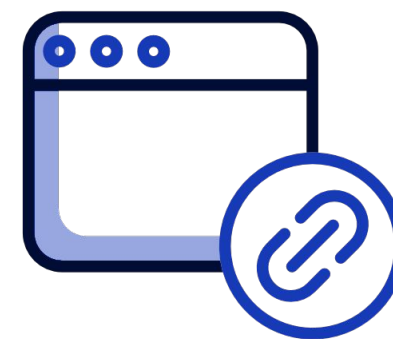
Работа с идентификатором ресурса

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api', methods=['POST'])
def parse_json():
    json_data = request.get_json()
    # Do something with the json_data
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run()
```



Необходимо указать **URL**

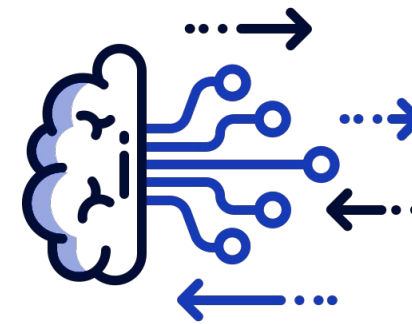
Работа с различными данными

```
from flask import Flask, request, jsonify
import xml.etree.ElementTree as ET

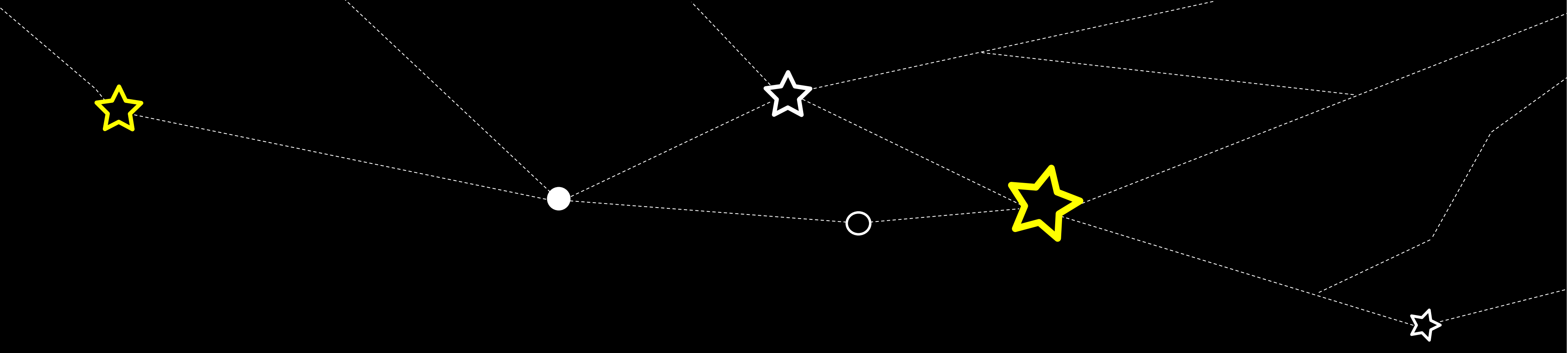
app = Flask(__name__)

@app.route('/api', methods=['POST'])
def parse_xml():
    xml_data = request.data
    root = ET.fromstring(xml_data)
    # Do something with the root element
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run()
```



Запрос не обязан быть
в формате **JSON**



СПАСИБО ЗА ВНИМАНИЕ

Александр Пушкарёв



testclubru



n\а



aqaguy



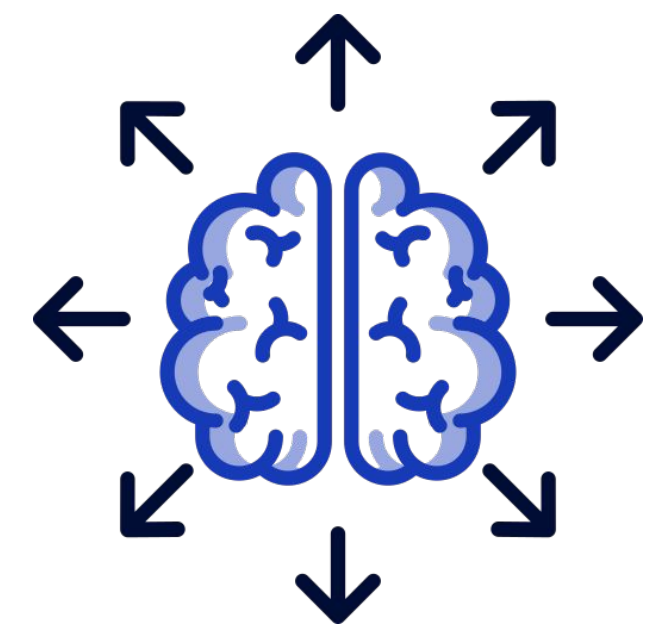
testclubru



aqaguy

Воркшоп

- 1 Необходимо разработать REST API,
предоставляющее возможность ведение блога
- 2 Сущности:
 - ★ Пользователь
 - ★ Твит
 - ★ Реакция на твит
- 3 Пользователь должен иметь возможность:
 - ★ Создать, прочитать, изменить и удалить твит
 - ★ Получить список всех твитов
 - ★ Добавить реакцию к твиту



Домашнее задание

- 1 Необходимо разработать REST API, предоставляющее возможность ведение блога
- 2 Сущности:
 - ★ Пользователь
 - ★ Пост
 - ★ Комментарий
- 3 Пользователь должен иметь возможность:
 - ★ Создать, прочитать, изменить и удалить пост
 - ★ Получить список всех постов
 - ★ Добавить и удалить комментарий к посту

